AI Assisted Coding

Lab Assignment 11.3

Name : V Sai Krishnachaithanya
Hall Ticket no : 2303A52154

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
o Add a contact
o Search for a contact
o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
o Insertion efficiency
o Deletion efficiency

Expected Outcome

• Two working implementations (array-based and linked-list-based).
• A brief comparison explaining performance differences.

Prompt :

Generate a python function for Contact Manager Application to store members' names and phone numbers. Implement the contact manager using arrays. Implement the same functionality using a linked list

```python
from typing import List, Tuple, Optional

# Contact Manager implementations: array-based and linked-list-based
# Save this cell as a new file or run in a notebook cell.


class ArrayContactManager:
    """Contact manager using Python lists (arrays)."""
    def __init__(self):
        self.contacts: List[Tuple[str, str]] = []  # list of (name, phone)

    def add_contact(self, name: str, phone: str) -> bool:
        """Add a contact. Returns False if exact duplicate exists."""
        entry = (name, phone)
        if entry in self.contacts:
            return False
        self.contacts.append(entry)
        return True
```

```python
    def delete_contact(self, name: str) -> int:
        """Delete contacts whose name contains the given string (case-insensitive).
        Returns number of deleted contacts."""
        key = name.lower()
        dummy = _Node("", "")
        dummy.next = self.head
        prev = dummy
        cur = self.head
        deleted = 0
        while cur:
            if key in cur.name.lower():
                prev.next = cur.next
                deleted += 1
                cur = prev.next
            else:
                prev = cur
                cur = cur.next
        self.head = dummy.next
        return deleted


# Example usage (for quick manual testing in the notebook)
if __name__ == "__main__":
    a = ArrayContactManager()
    a.add_contact("Alice Smith", "555-0101")
    a.add_contact("Bob Jones", "555-0202")
    print("Array search 'Alice':", a.search_contact("Alice"))
    print("Array delete 'Bob':", a.delete_contact("Bob"), "deleted")
    print("Array contacts:", a.contacts)

    l = LinkedListContactManager()
    l.add_contact("Alice Smith", "555-0101")
    l.add_contact("Bob Jones", "555-0202")
    print("Linked search 'Alice':", l.search_contact("Alice"))
    print("Linked delete 'Bob':", l.delete_contact("Bob"), "deleted")
    print("Linked search '':", l.search_contact(""))  # list all
```

Output :

```
✓ 0.0s

Array search 'Alice': [('Alice Smith', '555-0101')]
Array delete 'Bob': 1 deleted
Array contacts: [('Alice Smith', '555-0101')]
Linked search 'Alice': [('Alice Smith', '555-0101')]
Linked delete 'Bob': 1 deleted
Linked search '': [('Alice Smith', '555-0101')]
```

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

Prompt:

Generate a python code for Library manages book borrow requests, but faculty requests must be prioritized over student requests.Implement a Queue (FIFO) to manage book requests.add endequeue and dequeue methods

```python
from collections import deque
class LibraryQueue:
    def __init__(self):
        self.queue = deque()
    def enqueue(self, name, book, role):
        request = (name, book, role)
        self.queue.append(request)
        print("Request added:", request)
    def dequeue(self):
        if not self.queue:
            print("No requests in queue")
            return
        request = self.queue.popleft()
        print("Processing request:", request)
    def display(self):
        print("\nQueue Requests:")
        for req in self.queue:
            print(req)
import heapq
class LibraryPriorityQueue:
    def __init__(self):
        self.pq = []
    def enqueue(self, name, book, role):
        # Faculty gets higher priority
        if role.lower() == "faculty":
            priority = 1
        else:
            priority = 2
        heapq.heappush(self.pq, (priority, name, book, role))
        print("Request added:", name, book, role)
    def dequeue(self):
        if not self.pq:
            print("No requests")
            return
        request = heapq.heappop(self.pq)
        print("Processing:", request[1], "-", request[2], "-", request[3])
    def display(self):
        print("\nPriority Queue Requests:")
        for req in sorted(self.pq):
            print(req[1], "-", req[2], "-", req[3])
```

Output :

```
✓ 0.0s

NORMAL QUEUE SYSTEM
Request added: ('Rahul', 'Data Structures', 'Student')
Request added: ('Anita', 'Machine Learning', 'Faculty')
Request added: ('Kiran', 'Python Programming', 'Student')

Queue Requests:
('Rahul', 'Data Structures', 'Student')
('Anita', 'Machine Learning', 'Faculty')
('Kiran', 'Python Programming', 'Student')
Processing request: ('Rahul', 'Data Structures', 'Student')
Processing request: ('Anita', 'Machine Learning', 'Faculty')

Queue Requests:
('Kiran', 'Python Programming', 'Student')

PRIORITY QUEUE SYSTEM
Request added: Rahul Data Structures Student
Request added: Anita Machine Learning Faculty
Request added: Kiran Python Programming Student
Request added: Dr. Rao Artificial Intelligence Faculty

Priority Queue Requests:
Anita - Machine Learning - Faculty
Dr. Rao - Artificial Intelligence - Faculty
Kiran - Python Programming - Student
...

Priority Queue Requests:
Kiran - Python Programming - Student
Rahul - Data Structures - Student
```

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
o push(ticket)
o pop()
o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
o Checking whether the stack is empty
o Checking whether the stack is full (if applicable)

Expected Outcome

• Functional stack-based ticket management system.
• Clear demonstration of LIFO behavior.

Prompt :

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach. Analyze the above senario and implement Stack to manage support tickets and provide Push, Pop and Peek operations

```python
#SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially,
# Analyze the above senario and implement Stack to manage support tickets and provide Push, Pop and Peek operations
class ITSupportStack:
    def __init__(self):
        self.stack = []
    def push(self, ticket_id, issue):
        ticket = (ticket_id, issue)
        self.stack.append(ticket)
        print("Ticket added:", ticket)
    def pop(self):
        if not self.stack:
            print("No tickets to process")
            return
        ticket = self.stack.pop()
        print("Processing ticket:", ticket)
    def peek(self):
        if not self.stack:
            print("No tickets in stack")
            return
        ticket = self.stack[-1]
        print("Next ticket to process:", ticket)
# Example usage
it_stack = ITSupportStack()
it_stack.push("T001", "Cannot connect to Wi-Fi")
it_stack.push("T002", "Laptop won't start")
it_stack.peek()  # Should show T002
it_stack.pop()   # Should process T002
it_stack.peek()  # Should show T001
it_stack.pop()   # Should process T001
it_stack.pop()   # Should indicate no tickets left
```

Output :

```
✓ 0.0s

Ticket added: ('T001', 'Cannot connect to Wi-Fi')
Ticket added: ('T002', "Laptop won't start")
Next ticket to process: ('T002', "Laptop won't start")
Processing ticket: ('T002', "Laptop won't start")
Next ticket to process: ('T001', 'Cannot connect to Wi-Fi')
Processing ticket: ('T001', 'Cannot connect to Wi-Fi')
No tickets to process
```

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Expected Outcome

• Collision handling using chaining

• Well-commented methods

Prompt :

Generate a python code to implement a Hash Table and understand collision handling. and provide insert, search and delete operations.

```python
# Generate a python code to implement a Hash Table and understand collision handling. and provide insert, search and delete operations.
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # list of buckets

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)  # Update existing key
                print(f"Updated key: {key} with value: {value}")
                return
        bucket.append((key, value))  # Add new key-value pair
        print(f"Inserted key: {key} with value: {value}")

    def search(self, key):
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                print(f"Found key: {key} with value: {v}")
                return v
        print(f"Key: {key} not found")
        return None

    def delete(self, key):
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                print(f"Deleted key: {key}")
                return True
        print(f"Key: {key} not found for deletion")
        return False
```

Ouput :

```
✓  0.0s

Inserted key: name with value: Alice
Inserted key: age with value: 30
Found key: name with value: Alice
Found key: age with value: 30
Deleted key: name
Key: name not found
```

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

• Student Attendance Tracking
• Event Registration System
• Library Book Borrowing
• Bus Scheduling System
• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

• Mapping table: Feature → Data Structure → Justification
• One fully working Python implementation

Prompt :

Design a Campus Resource Management System with the following features: Student Attendance Tracking,Event Registration System,Library Book Borrowing,Bus Scheduling System,Cafeteria Order Queue.analyze the above senario and implement a simple system to manage these resources using appropriate data structures.

```python
#Design a Campus Resource Management System with the following features: Student Attendance Tracking,Event Registration System,Library Book Borrowing,Bus
# analyze the above senario and implement a simple system to manage these resources using appropriate data structures.
class CampusResourceManagement:
    def __init__(self):
        self.attendance = {}  # student_id: attendance_count
        self.events = {}      # event_name: set of registered student_ids
        self.library = {}     # book_title: borrower_id
        self.bus_schedule = []  # list of (bus_number, departure_time)
        self.cafeteria_queue = deque()  # queue of (student_id, order)

    # Student Attendance Tracking
    def mark_attendance(self, student_id):
        self.attendance[student_id] = self.attendance.get(student_id, 0) + 1
        print(f"Marked attendance for student {student_id}. Total: {self.attendance[student_id]}")

    # Event Registration System
    def register_event(self, student_id, event_name):
        if event_name not in self.events:
            self.events[event_name] = set()
        self.events[event_name].add(student_id)
        print(f"Student {student_id} registered for event '{event_name}'")

    # Library Book Borrowing
    def borrow_book(self, student_id, book_title):
        if book_title in self.library:
            print(f"Book '{book_title}' is already borrowed by student {self.library[book_title]}")
            return False
        self.library[book_title] = student_id
        print(f"Student {student_id} borrowed book '{book_title}'")
        return True

    def return_book(self, book_title):
        if book_title in self.library:
            borrower = self.library.pop(book_title)
            print(f"Book '{book_title}' returned by student {borrower}")
            return True
        print(f"Book '{book_title}' was not borrowed")
        return False
```

Output :

```
✓  0.0s

Marked attendance for student S001. Total: 1
Student S001 registered for event 'Tech Talk'
Student S001 borrowed book 'Data Structures'
Scheduled bus Bus1 at 08:00 AM
Student S001 placed order: Veg Sandwich

Bus Schedule:
Bus Bus1 departs at 08:00 AM
Processing order for student S001: Veg Sandwich
Book 'Data Structures' returned by student S001

True
```